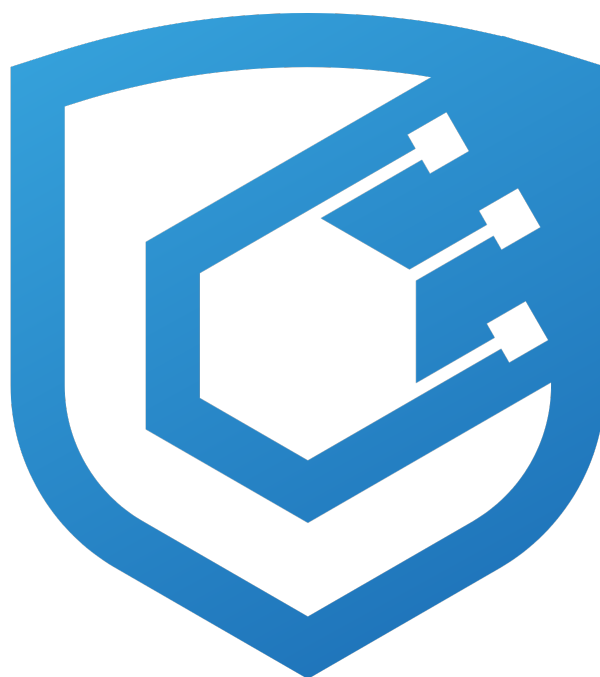




Foundations United BridgeX Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Jorge](#)

April 1, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	12
7.1	Medium Risk	12
7.1.1	PrivateChainBridge, PublicBridge::lockTokens allows locking tokens to unconfigured destination chains, permanently losing user funds	12
7.2	Low Risk	13
7.2.1	Use low-level call to send ETH if return data is unimportant to avoid return bomb DoS and for gas efficiency	13
7.2.2	PublicBridge, PrivateChainBridge::payReleaseFee will revert from out-of-gas if too many releasers are added	13
7.2.3	PublicBridge, PrivateChainBridge::removeReleaser can reduce releasers below requiredSignatures, making all release unprocessable	14
7.2.4	Constructors don't validate releaseFee > 0, causing payReleaseFee to panic with division by zero	15
7.2.5	Use Foundry's encrypted secure private key storage instead of plaintext environment variables	16
7.2.6	Pending Releases in PrivateChainBridge could be stuck	16
7.2.7	No way to recover or refund unused eligible releases in the bridges	17
7.3	Informational	19
7.3.1	Use named imports	19
7.3.2	Use named mapping parameters to explicitly denote the purpose of keys and values	19
7.3.3	Emit missing events for important state changes	19
7.3.4	Implement a reasonable hard-coded limit on maximum fees	19
7.3.5	Prefix private, internal function names with _ character	20
7.3.6	Use explicit uint256 instead of uint	20
7.3.7	Token::setIssuer allows setting issuer to address(0), disabling minting	20
7.3.8	PublicBridge::lockTokens allows zero-amount bridge operations when bridge fee is zero	21
7.3.9	Removed releaser sign still count	21
7.3.10	Sanity check correct chainId matches current chain in PublicBridge, PrivateChainBridge::_releaseTokens	22
7.4	Gas Optimization	23
7.4.1	Use immutable variables for non-upgradeable contracts whose storage is only set once in the constructor	23
7.4.2	In Solidity don't initialize to default values	23
7.4.3	Better storage packing	23
7.4.4	For reentrancy protection use transient keyword or ReentrancyGuardTransient instead of declaring a storage variable	24
7.4.5	Cache storage to avoid identical storage reads	24
7.4.6	In PublicBridge, PrivateChainBridge::payReleaseFee don't iterate over releasers array when feePerReleaser is zero	25
7.4.7	Use input value instead of defining additional variable when original input value doesn't need to be preserved	25
7.4.8	Use msg.sender instead of owner inside onlyOwner functions	25
7.4.9	Use input parameter when emitting events instead of reading storage when value already known	25

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

BridgeX is a bi-directional bridge that wraps native gas tokens on private GoQuorum-based blockchains ("brand chains") into ERC20 representations on public EVM chains (Ethereum, BSC, and Base). Users earn gas tokens on the private chain, then bridge (wrap) them to public chains where the newly minted ERC20 supply can be used to seed DEX/CEX liquidity pools.

The system consists of three contracts:

- `PrivateChainBridge` — Deployed on the private GoQuorum chain. Handles locking and releasing of native gas tokens via a `TokenVault`. When bridging outbound (private to public), users lock native tokens; when bridging inbound (public to private), relayers authorize the release of native tokens from the vault.
- `PublicBridge` — Deployed on each public EVM chain. Handles locking and releasing of `Token` ERC20 tokens via a `TokenVault`. When bridging outbound (public to private), users lock `Token` ERC20 tokens in the vault; when bridging inbound (private to public), tokens are released from the vault or minted if the vault has insufficient balance. The bridge holds the `issuer` role on the `Token` contract, allowing it to mint new supply up to `maxSupply`.
- `Token` — A standard ERC20 contract deployed on each public chain. Represents the wrapped form of the private chain's native gas token. Supply is controlled by the bridge via a permissioned `mint` function with a hard `maxSupply` cap. Ownership uses a two-step transfer pattern.

The bridge uses a configurable multi-signature relayer model: independent relayer nodes monitor `TokensLocked` events on the source chain and independently call `signRelease` on the destination chain. Tokens are only released once both relayers have signed. Releases require users to pre-pay a `releaseFee` in the destination chain's native token which is distributed among relayers. Users who have not pre-paid receive pending releases that are automatically processed upon fee payment.

A `bridgeFee` denominated in the bridged token is deducted on the source chain during locking and sent to a configurable `bridgeFeeReceiver` address. The bridge supports pausing, owner-controlled fee adjustments, and an emergency `forceProcessPendingRelease` function for owner intervention. A `DestinationChain` enum with reserved values and a configurable `chainIds` mapping allow extending to additional chains without redeployment.

5 Audit Scope

The audit scope was limited to:

```
src/PrivateChainBridge.sol
src/PublicBridge.sol
src/Token.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [BridgeX](#) smart contracts provided by [Foundations United](#). The findings consist of 1 Medium & 7 Low severity issues with the remainder being informational and gas optimizations.

Protocol Invariants

As part of our protocol analysis we developed the following proposed protocol invariants:

Token:

1. The sum of all individual ``balanceOf`` values must equal ``totalSupply`` (sum-of-balances conservation)
2. ``totalSupply`` never exceeds ``maxSupply``
3. ``totalSupply`` only increases (there is no burn function)
4. Only the ``issuer`` can mint new tokens
5. Only the ``owner`` can change the ``issuer``; ``setIssuer`` rejects ``address(0)`` (broken by I-7) (fixed ↪ after mitigation review)
6. Only the ``pendingOwner`` can confirm an ownership transfer
7. Token transfers conserve value: ``balanceOf[from]`` decreases and ``balanceOf[to]`` increases by the same ↪ amount, with no net creation or destruction
8. Transfers to ``address(0)`` always revert
9. Minting to ``address(0)`` always reverts

PublicBridge & PrivateChainBridge (shared invariants)

Releaser array-mapping consistency:

10. An address has ``isReleaser[addr] == true`` if and only if that address exists in the ``releasers`` ↪ array; the array and mapping are always in sync
11. The ``releasers`` array contains no duplicates
12. ``requiredSignatures`` is always between 1 and ``releasers.length`` inclusive (broken by L-3)

Multi-signature system:

13. ``signatureCount[txHash]`` equals the number of releasers for which ``signatures[txHash][releaser] == true`` (aggregation consistency) ↪
14. ``signatureCount[txHash]`` only increases; individual signatures (``signatures[txHash][releaser]``) once set to true never revert to false (monotonicity) ↪
15. A ``sourceChainTxHash`` once marked as processed (``processedHashes[sourceChainTxHash] == true``) never reverts to false (irreversibility) ↪
16. Release execution requires at least ``requiredSignatures`` distinct releaser signatures before tokens are released or a pending release is created ↪
17. The same releaser cannot sign the same release transaction hash more than once

Release fee eligibility:

18. ``eligibleBridgeReleases[user]`` only increases via ``payReleaseFee`` (by ``msg.value / releaseFee``) and ↪ only decreases by 1 per successful release or pending-release processing; no other function modifies it ↪
19. A release is executed immediately only if the recipient has ``eligibleBridgeReleases > 0``; otherwise ↪ it goes to pending state

Pending release system:

- 20. ``hasPendingReleases[addr] == true`` if and only if ``pendingReleasesByRecipient[addr]`` contains at least one entry with ``exists == true``
- 21. An address appears in the ``pendingRecipients`` array if and only if ``hasPendingReleases[addr] == true`` (array-mapping synchronization)
- 22. Pending releases are processed in FIFO order; each processed release has its ``exists`` flag set to `false``
- 23. ``forceProcessPendingRelease`` can only be called by the ``owner`` and bypasses fee eligibility checks

Access control:

- 24. Only the ``owner`` can add/remove releasers, set required signatures, set fees, set fee receiver, pause, unpause, transfer ownership, and force-process pending releases
- 25. Only addresses with ``isReleaser[msg.sender] == true`` can call ``signRelease``
- 26. When ``paused == true``, the functions ``lockTokens``, ``signRelease``, and ``payReleaseFee`` all revert
- 27. Only the ``pendingOwner`` can call ``acceptOwnership`` to complete a two-step ownership transfer

Fee correctness:

- 28. ``releaseFee`` is always greater than zero (enforced by ``setReleaseFee``)
- 29. ``bridgeFeeReceiver`` is never ``address(0)`` (enforced at construction and by ``setBridgeFeeReceiver``)
- 30. During ``lockTokens``, the locked amount plus the bridge fee equals the total input: ``bridgeAmount + bridgeFee == amount`` (for ``PublicBridge`` ERC-20) or ``bridgeAmount + bridgeFee == msg.value`` (for ``PrivateChainBridge`` native token)

Native token conservation:

- 37. ``payReleaseFee`` ETH conservation: the total native token value distributed to releasers plus the refund to the caller equals ``msg.value`` no ETH is created, destroyed, or retained by the bridge during fee processing
- 38. No ETH accumulation: after any successful state-changing function call, neither bridge contract's native token balance increases the contracts have no ``receive``/``fallback`` functions and all incoming ETH is forwarded to its intended recipients (releasers, vault, bridge fee receiver, or refunded to caller)

Vault authorization:

- 31. Only the corresponding bridge contract can call ``vault::release``; all other callers revert

Nonce monotonicity:

- 32. ``_nonce`` only increases (incremented by 1 on each ``lockTokens`` call, never decremented or reset)

PublicBridge-specific:

- 33. When the vault has insufficient ERC-20 token balance to cover a release, the bridge attempts to mint the shortfall from the ``Token`` contract; if minting fails (e.g., ``maxSupply`` reached), the release returns false and the release is added to pending state rather than reverting
- 34. ``lockTokens`` only accepts ``destinationChain == Private`` and ``amount > 0``; all other destination chains or zero amounts revert (broken by I-8) (fixed after mitigation review)

PrivateChainBridge-specific:

- 35. ``lockTokens`` reverts if ``destinationChain == Private`` (cannot bridge to self), if ``msg.value == 0``, or if the destination chain is not configured (``chainId == 0``) (broken by M-1) (fixed after mitigation review)
- 36. ``_releaseTokens`` validates ``destinationChain == Private`` and returns ``false`` (rather than reverting) if the vault's native token balance is less than the release amount, allowing the release to be added to pending state for later retry

Immutability:

39-41. ``vault`` (both bridges), ``token`` (PublicBridge), ``maxSupply`` and ``decimals`` (Token) were declared
 ↳ ``immutable`` as part of mitigations, enforcing immutability at the compiler level. No runtime
 ↳ verification needed
 42. Token's ``name`` and ``symbol`` are not modified by any function after construction (strings cannot be
 ↳ declared ``immutable`` in Solidity 0.8.18)

State variable modification authorization:

43. ``owner`` only changes via ``acceptOwnership`` in both bridge contracts (and via ``confirmOwnership`` in
 ↳ ``Token``); no other function modifies it
 44. ``pendingOwner`` only changes via ``transferOwnership`` (which sets it) and
 ↳ ``acceptOwnership`/`confirmOwnership`` (which clears it to ``address(0)``); no other function modifies
 ↳ it
 45. ``paused`` only changes via ``pause`` (sets to true) and ``unpause`` (sets to false); no other function
 ↳ modifies it
 46. ``releaseFee`` only changes via ``setReleaseFee``; no other function modifies it
 47. ``bridgeFee`` only changes via ``setBridgeFee``; no other function modifies it
 48. ``bridgeFeeReceiver`` only changes via ``setBridgeFeeReceiver``; no other function modifies it
 49. ``requiredSignatures`` only changes via ``setRequiredSignatures``; no other function modifies it
 50. ``chainIds`` mapping only changes via ``setChainId``; no other function modifies it
 51. ``processedHashes`` only transitions from false to true via ``signRelease``; no other function modifies
 ↳ it

Function post-condition integrity:

52. ``addReleaser`` integrity: after a successful ``addReleaser(addr)``, ``isReleaser[addr] == true`` and
 ↳ ``releasers.length`` has increased by exactly 1
 53. ``removeReleaser`` integrity: after a successful ``removeReleaser(addr)``, ``isReleaser[addr] == false``
 ↳ and ``releasers.length`` has decreased by exactly 1
 54. ``acceptOwnership`/`confirmOwnership`` integrity: after a successful call, ``owner`` equals the previous
 ↳ ``pendingOwner`` value, and ``pendingOwner`` is ``address(0)``
 55. ``transferFrom`` allowance correctness: after a successful ``transferFrom(from, to, value)``,
 ↳ ``allowance[from][msg.sender]`` has decreased by exactly ``value``

Structural:

56. The ``pendingRecipients`` array contains no duplicate addresses

Deployment Bytecode Verification

At the conclusion of the audit after all mitigations were successfully verified, Cyfrin was asked to verify that at the final mitigation commit [8153e1e](#) the locally compiled bytecode matched the bytecode deployed to Ethereum mainnet. We used the following reproducible process to do this.

1) prepare the repo:

```
gh repo clone NerdUnited-NodeGovernance/bridge-x-contracts
cd bridge-x-contracts
git checkout 8153e1ee1002cb440cf165a73adc3177289e27c4
forge install
forge build
forge test
```

2) prepare the helper programs:

To extract deployment bytecode use this python program `extract_deployed_bytecode.py`:

```
import sys
import os
import requests

if len(sys.argv) != 4:
```

```

    print("Usage: python3 extract_deployed_bytecode.py <env_var_for_rpc_url> <address> <output_path>")
    sys.exit(1)

env_var = sys.argv[1]
address = sys.argv[2]
output_path = sys.argv[3]

rpc_url = os.environ.get(env_var)
if not rpc_url:
    print(f"Error: Environment variable '{env_var}' is not set.")
    sys.exit(1)

data = {
    "jsonrpc": "2.0",
    "method": "eth_getCode",
    "params": [address, "latest"],
    "id": 1
}

response = requests.post(rpc_url, json=data)
if response.status_code == 200:
    result = response.json()
    if "result" in result:
        bytecode = result["result"]
        with open(output_path, "w") as f:
            f.write(bytecode)
        print(f"Deployed bytecode saved to {output_path}")
    else:
        print("Error: No bytecode found in response")
else:
    print(f"Error: RPC request failed with status {response.status_code}")

```

To compare locally compiled bytecode against deployed bytecode ignoring differences in metadata & immutable positions, use this python program `compare_bytecodes.py`:

```

import sys
import json
import os

def strip_metadata(bytecode: bytes) -> bytes:
    if len(bytecode) < 2:
        return bytecode
    cbor_len = (bytecode[-2] << 8) | bytecode[-1]
    if cbor_len <= 0 or cbor_len + 2 > len(bytecode):
        return bytecode
    # Optional validation: Check if it looks like CBOR metadata (starts with 0xa1 or 0xa2)
    potential_cbor = bytecode[-cbor_len - 2 : -2]
    if potential_cbor[0] not in (0xa1, 0xa2): # Common for Solidity metadata
        return bytecode
    return bytecode[:-cbor_len - 2]

if len(sys.argv) != 3:
    print("Usage: python3 compare_bytecodes.py <artifact_json_path> <deployed_bytecode_path>")
    sys.exit(1)

artifact_path = sys.argv[1]
deployed_path = sys.argv[2]

try:
    with open(artifact_path, "r") as f:
        data = json.load(f)

    # Get local deployed bytecode

```

```

deployed_bytecode_section = data.get("deployedBytecode")
if isinstance(deployed_bytecode_section, str):
    local_hex = deployed_bytecode_section
elif isinstance(deployed_bytecode_section, dict):
    local_hex = deployed_bytecode_section.get("object")
else:
    print("Error: 'deployedBytecode' not found or invalid in artifact.")
    sys.exit(1)

if not local_hex:
    print("Error: No bytecode found in artifact.")
    sys.exit(1)

# Strip '0x' if present
if local_hex.startswith("0x"):
    local_hex = local_hex[2:]

local_bytes = bytes.fromhex(local_hex)

# Get deployed bytecode from file
with open(deployed_path, "r") as f:
    deployed_hex = f.read().strip()

if deployed_hex.startswith("0x"):
    deployed_hex = deployed_hex[2:]

deployed_bytes = bytes.fromhex(deployed_hex)

# Strip metadata from both
local_bytes = strip_metadata(local_bytes)
deployed_bytes = strip_metadata(deployed_bytes)

if len(local_bytes) != len(deployed_bytes):
    print(f"Error: Bytecode lengths differ after stripping metadata: local {len(local_bytes)} vs
    ↳ deployed {len(deployed_bytes)}")
    sys.exit(1)

# Compare, ignoring positions where local is 0x00 but deployed is not (assumed to be immutable
    ↳ placeholders)
mismatch = False
assumed_immutable_positions = []
for i in range(len(local_bytes)):
    if local_bytes[i] == 0 and deployed_bytes[i] != 0:
        assumed_immutable_positions.append(i)
        continue
    if local_bytes[i] != deployed_bytes[i]:
        print(f"Mismatch at position {i}: local 0x{local_bytes[i]:02x} vs deployed
        ↳ 0x{deployed_bytes[i]:02x}")
        mismatch = True

if mismatch:
    print("Bytecodes do NOT match (differences outside assumed immutable positions).")
else:
    print("Bytecodes match, ignoring differences in assumed immutable positions.")

if assumed_immutable_positions:
    print("\nAssumed immutable positions where local=0x00 and deployed !=0x00:")
    # Group consecutive positions
    groups = []
    start = assumed_immutable_positions[0]
    prev = assumed_immutable_positions[0]
    for pos in assumed_immutable_positions[1:]:
        if pos == prev + 1:

```



```

        prev = pos
    else:
        groups.append((start, prev))
        start = pos
        prev = pos
    groups.append((start, prev))
    for s, e in groups:
        length = e - s + 1
        local_val = local_bytes[s:s+length].hex()
        deployed_val = deployed_bytes[s:s+length].hex()
        print(f" Range {s}-{e} (length {length}): local {local_val} vs deployed {deployed_val}")

except FileNotFoundError as e:
    print(f"Error: File not found - {e}")
    sys.exit(1)

```

3) verify Token deployed bytecode using:

- python3 extract_deployed_bytecode.py ETH_RPC_URL 0x1ae1a1130e581240a9ed81355312fdb2e741f100 deployments/bytecode/Token.mainnet.txt to extract mainnet deployed bytecode
- python3 compare_bytecodes.py out/Token.sol/Token.json deployments/bytecode/Token.mainnet.txt to verify both match - succeeds with these differences where local=0x00 and deployed!=0x00 but are known immutable values:

```

Assumed immutable positions where local=0x00 and deployed !=0x00:
Range 1388-1388 (length 1): local 00 vs deployed 12 (decimals = 18)
Range 1561-1569 (length 9): local 000000000000000000 vs deployed a18f07d736b90be550 (maxSupply =
↳ 50,000,000,000 * 1018)

```

4) verify PublicBridge deployed bytecode using:

- python3 extract_deployed_bytecode.py ETH_RPC_URL 0x544918caf667b200936718e51094800a959e7ffc deployments/bytecode/PublicBridge.mainnet.txt to extract mainnet deployed bytecode
- python3 compare_bytecodes.py out/PublicBridge.sol/PublicBridge.json deployments/bytecode/PublicBridge.mainnet.txt to verify both match - succeeds with these differences where local=0x00 and deployed!=0x00 but are known immutable values:

```

Assumed immutable positions where local=0x00 and deployed !=0x00:
Range 10832-10850 (length 19): local 00000000000000000000000000000000 vs deployed
↳ 1ae1a1130e581240a9ed81355312fdb2e741f1 (Token)
Range 11158-11176 (length 19): local 00000000000000000000000000000000 vs deployed
↳ 1ae1a1130e581240a9ed81355312fdb2e741f1 (Token)
Range 11219-11238 (length 20): local 00000000000000000000000000000000 vs deployed
↳ 9885ee74cb9ebdd2835ca2867c1a2877022c6dba (TokenVault)
Range 12802-12821 (length 20): local 00000000000000000000000000000000 vs deployed
↳ 9885ee74cb9ebdd2835ca2867c1a2877022c6dba (TokenVault)
Range 12838-12856 (length 19): local 00000000000000000000000000000000 vs deployed
↳ 1ae1a1130e581240a9ed81355312fdb2e741f1 (Token)
Range 13396-13414 (length 19): local 00000000000000000000000000000000 vs deployed
↳ 1ae1a1130e581240a9ed81355312fdb2e741f1 (Token)
Range 13456-13475 (length 20): local 00000000000000000000000000000000 vs deployed
↳ 9885ee74cb9ebdd2835ca2867c1a2877022c6dba (TokenVault)
Range 13591-13610 (length 20): local 00000000000000000000000000000000 vs deployed
↳ 9885ee74cb9ebdd2835ca2867c1a2877022c6dba (TokenVault)
Range 13753-13771 (length 19): local 00000000000000000000000000000000 vs deployed
↳ 1ae1a1130e581240a9ed81355312fdb2e741f1 (Token)
Range 13990-14009 (length 20): local 00000000000000000000000000000000 vs deployed
↳ 9885ee74cb9ebdd2835ca2867c1a2877022c6dba (TokenVault)

```

Note: the Token immutable address 0x1ae1a1130e581240a9ed81355312fdb2e741f100 has a trailing 00 byte which matches the local zero-initialized bytecode, hence only 19 of its 20 address bytes appear as differences. The

TokenVault at address 0x9885ee74cb9ebdd2835ca2867c1a2877022c6dba was deployed internally by the PublicBridge constructor. The token immutable appears 5 times and the vault immutable appears 5 times in the bytecode, consistent with multiple references to these immutables throughout the contract code.

5) verify BSC and Base mainnet deployments:

The client confirmed that the same contracts were deployed to BSC and Base mainnets at the same addresses. We repeated the extraction and comparison using public RPC endpoints (<https://bsc-dataseed.binance.org> and <https://mainnet.base.org>):

- `python3 extract_deployed_bytecode.py BSC_RPC_URL 0x1ae1a1130e581240a9ed81355312fdb2e741f100 deployments/bytecode/Token.bsc.txt`
- `python3 compare_bytecodes.py out/Token.sol/Token.json deployments/bytecode/Token.bsc.txt` - succeeds, identical immutable differences as Ethereum mainnet
- `python3 extract_deployed_bytecode.py BSC_RPC_URL 0x544918caf667b200936718e51094800a959e7ffc deployments/bytecode/PublicBridge.bsc.txt`
- `python3 compare_bytecodes.py out/PublicBridge.sol/PublicBridge.json deployments/bytecode/PublicBridge.bsc.txt` - succeeds, identical immutable differences as Ethereum mainnet
- `python3 extract_deployed_bytecode.py BASE_RPC_URL 0x1ae1a1130e581240a9ed81355312fdb2e741f100 deployments/bytecode/Token.base.txt`
- `python3 compare_bytecodes.py out/Token.sol/Token.json deployments/bytecode/Token.base.txt` - succeeds, identical immutable differences as Ethereum mainnet
- `python3 extract_deployed_bytecode.py BASE_RPC_URL 0x544918caf667b200936718e51094800a959e7ffc deployments/bytecode/PublicBridge.base.txt`
- `python3 compare_bytecodes.py out/PublicBridge.sol/PublicBridge.json deployments/bytecode/PublicBridge.base.txt` - succeeds, identical immutable differences as Ethereum mainnet

All deployments across Ethereum, BSC, and Base mainnets are byte-for-byte identical (including constructor arguments and immutable values), confirming the same audited code is deployed on all three chains.

Summary

Project Name	BridgeX
Repository	bridge-x-contracts
Commit	406c951d3b3e...
Fix Commit	8153e1ee1002...
Audit Timeline	Mar 3rd - Mar 9th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	7
Informational	10
Gas Optimizations	9
Total Issues	27

Summary of Findings

[M-1] PrivateChainBridge, PublicBridge::lockTokens allows locking tokens to unconfigured destination chains, permanently losing user funds	Resolved
[L-1] Use low-level call to send ETH if return data is unimportant to avoid return bomb DoS and for gas efficiency	Resolved
[L-2] PublicBridge, PrivateChainBridge::payReleaseFee will revert from out-of-gas if too many releasers are added	Acknowledged
[L-3] PublicBridge, PrivateChainBridge::removeReleaser can reduce releasers below requiredSignatures, making all release unprocessable	Acknowledged
[L-4] Constructors don't validate releaseFee > 0, causing payReleaseFee to panic with division by zero	Resolved
[L-5] Use Foundry's encrypted secure private key storage instead of plaintext environment variables	Resolved
[L-6] Pending Releases in PrivateChainBridge could be stuck	Resolved
[L-7] No way to recover or refund unused eligible releases in the bridges	Acknowledged
[I-1] Use named imports	Resolved
[I-2] Use named mapping parameters to explicitly denote the purpose of keys and values	Resolved
[I-3] Emit missing events for important state changes	Resolved
[I-4] Implement a reasonable hard-coded limit on maximum fees	Acknowledged
[I-5] Prefix private, internal function names with _ character	Resolved
[I-6] Use explicit uint256 instead of uint	Resolved
[I-7] Token::setIssuer allows setting issuer to address(0), disabling minting	Resolved
[I-8] PublicBridge::lockTokens allows zero-amount bridge operations when bridge fee is zero	Resolved
[I-9] Removed releaser sign still count	Acknowledged
[I-10] Sanity check correct chainId matches current chain in PublicBridge, PrivateChainBridge::_releaseTokens	Resolved

[G-1] Use <code>immutable</code> variables for non-upgradeable contracts whose storage is only set once in the constructor	Resolved
[G-2] In Solidity don't initialize to default values	Resolved
[G-3] Better storage packing	Resolved
[G-4] For reentrancy protection use <code>transient</code> keyword or <code>ReentrancyGuard-Transient</code> instead of declaring a storage variable	Acknowledged
[G-5] Cache storage to avoid identical storage reads	Resolved
[G-6] In <code>PublicBridge</code> , <code>PrivateChainBridge::payReleaseFee</code> don't iterate over <code>releasers</code> array when <code>feePerReleaser</code> is zero	Resolved
[G-7] Use input value instead of defining additional variable when original input value doesn't need to be preserved	Resolved
[G-8] Use <code>msg.sender</code> instead of <code>owner</code> inside <code>onlyOwner</code> functions	Resolved
[G-9] Use input parameter when emitting events instead of reading storage when value already known	Resolved

7 Findings

7.1 Medium Risk

7.1.1 PrivateChainBridge, PublicBridge::lockTokens allows locking tokens to unconfigured destination chains, permanently losing user funds

Description: PrivateChainBridge::lockTokens only validates that the destination chain is not Private:

```
// PrivateChainBridge.sol:425-426
require(msg.value > 0, "Amount required");
require(destinationChain != DestinationChain.Private, "Cannot bridge to self");
```

The DestinationChain enum includes Reserved4 through Reserved8 (values 4-8) intended for future chain expansion via setChainId. However, a user can call lockTokens with any of these Reserved values before the owner configures them. The tokens are locked in the vault, the TokensLocked event emits with chainId = 0 (unconfigured default), and relayers have no corresponding destination bridge to process the release.

```
// PrivateChainBridge.sol:443-461
uint256 chainId = getChainId(destinationChain); // returns 0 for unconfigured chains

emit TokensLocked(
    msg.sender,
    bridgeAmount,
    uniqueHash,
    destinationChain,
    chainId // 0 - relayers cannot route this
);
```

The locked tokens cannot be recovered — the TokenVault has no sweep function and only releases tokens via the bridge's _releaseTokens path, which requires relayer signatures from the non-existent destination chain.

Impact: Native tokens locked to unconfigured destination chains are irrecoverable; users permanently lose their funds.

Recommended Mitigation: Validate that the destination chain has a configured chain ID:

```
function lockTokens(DestinationChain destinationChain) external payable nonReentrant whenNotPaused {
    require(msg.value > 0, "Amount required");
    require(destinationChain != DestinationChain.Private, "Cannot bridge to self");
    uint256 destChainId = getChainId(destinationChain);
    require(destChainId != 0, "Destination chain not configured");
    // ... use destChainId instead of calling getChainId again
}
```

The same fix is required for PublicBridge::lockTokens, or alternatively just emit 0 for the chainId if 0 is valid value for the private chain.

BridgeX: Fixed in commits [ab40545](#), [3320c2b](#), [32882e0](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 Use low-level call to send ETH if return data is unimportant to avoid return bomb DoS and for gas efficiency

Description: Use low-level call to send ETH if return data is unimportant, for example in `PublicBridge::payReleaseFee`:

```
-      (bool success, ) = payable(releasers[i]).call{value: feePerReleaser}("");
+      bool success;
+      address feeReceiver = releasers[i];

+      assembly {
+          success := call(gas(), feeReceiver, feePerReleaser, 0, 0, 0, 0)
+      }
```

This avoids the "return-bomb" DoS attack and is also more gas efficient. All affected places:

```
PublicBridge.sol
392:      (bool success, ) = payable(releasers[i]).call{value: feePerReleaser}("");
402:      (bool success, ) = payable(releasers[0]).call{value: remainingWei}("");
414:      (bool success, ) = payable(msg.sender).call{value: refundAmount}("");

PrivateChainBridge.sol
390:      (bool success, ) = payable(releasers[i]).call{value: feePerReleaser}("");
400:      (bool success, ) = payable(releasers[0]).call{value: remainingWei}("");
412:      (bool success, ) = payable(msg.sender).call{value: refundAmount}("");
```

BridgeX: Fixed in commit [efb6843](#).

Cyfrin: Verified.

7.2.2 PublicBridge, PrivateChainBridge::payReleaseFee will revert from out-of-gas if too many releasers are added

Description: `PublicBridge`, `PrivateChainBridge::payReleaseFee` uses a push-based pattern to distribute fees, iterating over the entire `releasers` array and sending native tokens to each releaser individually:

```
// PublicBridge.sol:390-397 (same pattern in PrivateChainBridge.sol:388-395)
uint256 distributed = 0;
for (uint i = 0; i < releasers.length; i++) {
    if (feePerReleaser > 0) {
        (bool success, ) = payable(releasers[i]).call{value: feePerReleaser}("");
        if (success) {
            distributed += feePerReleaser;
        }
    }
}
```

As acknowledged in the `README.md` (Known Issue 1 — releasers are trusted, Known Issue 7 — failed transfers are refunded to sender rather than held), this design intentionally skips failed transfers and refunds undistributed portions. However, the push pattern has two inherent drawbacks:

1. **Unbounded iteration:** Every call to `payReleaseFee` iterates the full `releasers` array. With the addition of Reserved enum values for future chain expansion, the number of releasers may grow. Gas cost scales linearly with the number of releasers, and at sufficient scale, `payReleaseFee` could exceed the block gas limit
2. **Partial failure discounts:** When some (but not all) releaser transfers fail, the undistributed portion is refunded to the caller while the full release credits are retained. For example, with 2 releasers and 1 failing, the user pays 50% of the intended fee but receives full release eligibility. The affected releaser receives no compensation for that release

A pull-based (withdrawal) pattern would address both concerns:

```

// Pull-based alternative
mapping(address releaserAddr => uint256 feeBalance) public releaserFeeBalance;

function payReleaseFee() external payable nonReentrant whenNotPaused {
    require(msg.value >= releaseFee, "Insufficient fee");
    require(releasers.length > 0, "No releasers available");

    uint256 eligibleReleases = msg.value / releaseFee;
    uint256 totalFee = eligibleReleases * releaseFee;

    // O(1) - no iteration over releasers
    uint256 feePerReleaser = totalFee / releasers.length;
    uint256 remainingWei = totalFee - (feePerReleaser * releasers.length);
    for (uint256 i; i < releasers.length; i++) {
        releaserFeeBalance[releasers[i]] += feePerReleaser;
    }
    if (remainingWei > 0) {
        releaserFeeBalance[releasers[0]] += remainingWei;
    }

    eligibleBridgeReleases[msg.sender] += eligibleReleases;

    // Refund only the excess beyond full release fees
    uint256 refundAmount = msg.value - totalFee;
    if (refundAmount > 0) {
        (bool success, ) = payable(msg.sender).call{value: refundAmount}("");
        require(success, "Refund failed");
    }

    _processPendingReleases(msg.sender);
}

function claimReleaserFees() external {
    uint256 amount = releaserFeeBalance[msg.sender];
    require(amount > 0, "No fees to claim");
    releaserFeeBalance[msg.sender] = 0;
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "Transfer failed");
}

```

This eliminates partial refund complexity, ensures full fee accounting regardless of individual releaser transfer success, and makes payReleaseFee O(1) for the fee acceptance path (the releaser iteration for balance updates remains but involves no external calls).

BridgeX: Acknowledged; we may implement a pull-based pattern in a subsequent version. For now the expected releaser set is small (2-5 addresses) so we will keep the current simple implementation.

7.2.3 PublicBridge, PrivateChainBridge::removeReleaser can reduce releasers below requiredSignatures, making all release unprocessable

Description: PublicBridge, PrivateChainBridge::removeReleaser does not validate that releasers.length remains at or above requiredSignatures after removal:

```

function removeReleaser(address _releaser) external onlyOwner {
    require(isReleaser[_releaser], "Not a releaser");
    isReleaser[_releaser] = false;
    for (uint i = 0; i < releasers.length; i++) {
        if (releasers[i] == _releaser) {
            releasers[i] = releasers[releasers.length - 1];
            releasers.pop();
            break;
        }
    }
}

```

```

    }
}
// missing: require(releasers.length >= requiredSignatures)
}

```

While `setRequiredSignatures` validates `_requiredSignatures <= releasers.length`, the reverse check is absent from `removeReleaser`. If the owner removes releasers to bring the count below `requiredSignatures`, the `signatureCount[txHash] >= requiredSignatures` condition in `signRelease` can never be met, making all in-progress and future releases unprocessable.

Impact:

- All releases become stuck until the owner either adds releasers back or calls `setRequiredSignatures` with a lower value
- In-progress releases that already have some signatures collected cannot reach the threshold
- Recoverable by the owner, but creates a window where the bridge is non-functional

Recommended Mitigation: Add a bounds check to `removeReleaser` in both bridges:

```

function removeReleaser(address _releaser) external onlyOwner {
    require(isReleaser[_releaser], "Not a releaser");
    uint256 releasersLength = releasers.length;
    require(releasersLength > requiredSignatures, "Would break signature threshold");
    // ... existing removal logic ...
}

```

BridgeX: Acknowledged; the owner role is trusted and would typically pause the bridge during relayer rotation. Adding a hard block on removal could hinder emergency response if a relayer is compromised. The owner can adjust `requiredSignatures` as needed.

7.2.4 Constructors don't validate `releaseFee > 0`, causing `payReleaseFee` to panic with division by zero

Description: Both bridge constructors accept `_releaseFee` without validating it is greater than zero:

```

// PublicBridge.sol:302-312
constructor(address _token, uint256 _releaseFee, uint256 _bridgeFee, address _bridgeFeeReceiver) {
    token = IERC20(_token);
    owner = msg.sender;
    releaseFee = _releaseFee; // no validation - allows 0
    // ...
}

// PrivateChainBridge.sol:282-296
constructor(uint256 _releaseFee, uint256 _bridgeFee, address _bridgeFeeReceiver) {
    owner = msg.sender;
    releaseFee = _releaseFee; // no validation - allows 0
    // ...
}

```

While `setReleaseFee` enforces `require(_newFee > 0, "Fee must be positive")`, the constructor has no equivalent check. If deployed with `releaseFee = 0`, the `payReleaseFee` function panics with a division by zero on its first line of computation:

```

// Both bridges: payReleaseFee()
uint256 eligibleReleases = msg.value / releaseFee; // Panic(0x12) when releaseFee == 0

```

Impact:

- `payReleaseFee` always panics with an opaque `Panic(0x12)` error instead of a meaningful revert message
- All releases go to pending state since no user can pay the release fee

- Only `forceProcessPendingRelease` (owner-only) can process releases
- Recoverable via `setReleaseFee`, but creates a confusing failure mode at deployment

Recommended Mitigation: Add validation in both constructors:

```
require(_releaseFee > 0, "Fee must be positive");
```

BridgeX: Fixed in commit [69b0e8e](#).

Cyfrin: Verified.

7.2.5 Use Foundry's encrypted secure private key storage instead of plaintext environment variables

Description: Both deployment scripts read the deployer's private key from a plaintext environment variable:

```
// DeployPrivateBridge.s.sol:10
uint256 deployerKey = vm.envUint("DEPLOYER_PRIVATE_KEY");

// DeployPublicBridge.s.sol:11
uint256 deployerKey = vm.envUint("DEPLOYER_PRIVATE_KEY");
```

This requires the private key to be stored in a `.env` file or shell environment as unencrypted plaintext. Plaintext private keys are exposed through version control leaks (accidental `.env` commits), shell history, misconfigured backups, compromised developer machines, or CI/CD logs. The deployer key is particularly sensitive as it becomes the owner of both bridge contracts with full admin privileges (pause, set fees, add/remove releasers, force-process releases).

Impact:

- Deployer private key exposure compromises the owner role on all deployed bridge contracts
- Attacker with the key can pause bridges, drain fees, manipulate releasers, or force-process releases
- Risk persists after deployment since the deployer key remains the owner

Recommended Mitigation: Use Foundry's encrypted keystore instead of plaintext environment variables:

```
# Import key once (encrypted with password, stored in ~/.foundry/keystores/)
cast wallet import deployer --interactive

# Use in deployment scripts (prompts for password, key never in plaintext)
forge script script/DeployPublicBridge.s.sol --account deployer --broadcast
```

Update deployment scripts to remove the private key parameter:

```
function run() external {
    // No private key in environment - Foundry decrypts from keystore at runtime
    vm.startBroadcast();
    // ... deployment logic ...
    vm.stopBroadcast();
}
```

See this Updraft [lesson](#) on using Foundry's encrypted secure private key storage.

BridgeX: Fixed in commit [caf269f](#).

Cyfrin: Verified.

7.2.6 Pending Releases in PrivateChainBridge could be stuck

Description: In `PrivateChainBridge`, pending releases are processed via `PrivateChainBridge::_processPendingReleases`, which calls `PrivateChainBridge::_releaseTokens` that reverts when the vault does not have enough native tokens (or the receiver can not receive native tokens because is an smart contract). Because

PrivateChainBridge::_releaseTokens has no return value and uses a require on vault balance, any insufficient balance condition (or revert inside vault::release) will revert the entire transaction, blocking processing of that user's pending releases and reverting their fee payment, in case that the loop already processed successful payment it will revert all of them. This contrasts with PublicBridge, where PublicBridge::_releaseTokens returns a bool and PublicBridge::_processPendingReleases handles failures gracefully by keeping the failing release pending and breaking the loop without reverting the whole transaction.

Private bridge release and processing:

```
function _processPendingReleases(address recipient) internal {
    PendingRelease[] storage releases = pendingReleasesByRecipient[recipient];

    for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
        if (releases[i].exists) {
            eligibleBridgeReleases[recipient]--;

            _releaseTokens(//@audit ok if a single token fail here then the rest of the releases
                → will be stuck
                payable(releases[i].recipient),
                releases[i].amount,
                releases[i].destinationChain
            );
        }
    }
}
```

```
function _releaseTokens(
    address payable to,
    uint256 amount,
    DestinationChain destinationChain
) internal {
    require(address(vault).balance >= amount, "Insufficient vault balance");

    vault.release(to, amount);

    uint256 chainId = getChainId(destinationChain);

    emit TokensReleased(to, amount, destinationChain, chainId);
}
```

Impact: If `address(vault).balance < amount` for any pending release in PrivateChainBridge, PrivateChainBridge::_releaseTokens reverts, causing the entire PrivateChainBridge::_processPendingReleases call to revert. The user's pending releases cannot be processed until the vault is fully funded for the first failing release, effectively blocking all the user's queued releases behind it.

Recommended Mitigation: Align PrivateChainBridge behavior with PublicBridge non-reverting pattern.

BridgeX: Fixed in commit [e390a56](#).

Cyfrin: Verified.

7.2.7 No way to recover or refund unused eligible releases in the bridges

Description: In the bridge contracts, users prepay the release fee via `Bridge::payReleaseFee` and receive a credit in `eligibleBridgeReleases[msg.sender]`. That credit is only consumed when pending releases are actually processed in `Bridge::_processPendingReleases`.

There is no function or flow that allows users to recover, refund, or transfer unused eligibility. If a user pays for more releases than they have pending the extra eligibility remains in the contract forever and cannot be turned back into value or given to someone else. Eligibility is only decremented when a pending release is processed (and incremented again only when that single release fails):

```
for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
    if (releases[i].exists) {
        eligibleBridgeReleases[recipient]--;
    }
}
```

```

    if (_releaseTokens(
        releases[i].recipient,
        releases[i].amount,
        releases[i].destinationChain
    )) {
        emit PendingReleaseProcessed(
            releases[i].recipient,
            releases[i].amount,
            releases[i].sourceChainTxHash
        );
        releases[i].exists = false;
    } else {
        eligibleBridgeReleases[recipient]++;
        break;
    }
}

```

There is no other code path that decreases or refunds `eligibleBridgeReleases` for unused slots.

Impact: Users who overpay or never get pending releases end up with unused eligibility that cannot be refunded, transferred, or otherwise used.

Recommended Mitigation: Add a function (e.g. `refundUnusedEligibility` or `refundEligibleReleases(uint256 count)`) that lets users surrender unused `eligibleBridgeReleases` to have their remaining fees refunded.

BridgeX: Acknowledged; a user's unused credits are consumed on future releases and refunds are impossible since fees are paid to relayers immediately. Adding a refund path would require the contract to hold funds introducing extra complexity and potential drain vectors.

7.3 Informational

7.3.1 Use named imports

Description: Use named imports:

```
PublicBridge.sol
- import "../Token.sol";
+ import {IERC20, Token} from "../Token.sol";
```

BridgeX: Fixed in commit [5b5523c](#).

Cyfrin: Verified.

7.3.2 Use named mapping parameters to explicitly denote the purpose of keys and values

Description: Use named mapping parameters to explicitly denote the purpose of keys and values:

```
PublicBridge.sol
79:     mapping(address => bool) public isReleaser;
100:    mapping(DestinationChain => uint256) public chainIds;
103:    mapping(bytes32 => bool) public processedHashes;
106:    mapping(address => uint256) public eligibleBridgeReleases;
109:    mapping(bytes32 => mapping(address => bool)) public signatures;
112:    mapping(bytes32 => uint256) public signatureCount;
129:    mapping(address => PendingRelease[]) public pendingReleasesByRecipient;
135:    mapping(address => bool) public hasPendingReleases;

PrivateChainBridge.sol
62:    mapping(DestinationChain => uint256) public chainIds;
65:    mapping(address => bool) public isReleaser;
94:    mapping(bytes32 => bool) public processedHashes;
97:    mapping(address => uint256) public eligibleBridgeReleases;
100:    mapping(bytes32 => mapping(address => bool)) public signatures;
103:    mapping(bytes32 => uint256) public signatureCount;
120:    mapping(address => PendingRelease[]) public pendingReleasesByRecipient;
126:    mapping(address => bool) public hasPendingReleases;

Token.sol
25:    mapping (address => uint256) public override balanceOf;
26:    mapping (address => mapping (address => uint256)) public override allowance;
```

BridgeX: Fixed in commit [43b99ff](#).

Cyfrin: Verified.

7.3.3 Emit missing events for important state changes

Description: Emit missing events for important state changes:

- PublicBridge, PrivateChainBridge::addReleaser, removeReleaser, setRequiredSignatures, setChainId

BridgeX: Fixed in commit [5899b57](#).

Cyfrin: Verified.

7.3.4 Implement a reasonable hard-coded limit on maximum fees

Description: Implement a reasonable hard-coded limit on maximum fees:

- PublicBridge, PrivateChainBridge::constructor, setReleaseFee, setBridgeFee

At the moment the contract owner could change the release fee to something unreasonable which would prevent users from releasing their tokens.

BridgeX: Acknowledged; may implement in a future version.

7.3.5 Prefix private, internal function names with _ character

Description: Prefix private, internal function names with _ character:

- Token::updateBalance

BridgeX: Fixed in commit [c2d98c6](#).

Cyfrin: Verified.

7.3.6 Use explicit uint256 instead of uint

Description: Use explicit uint256 instead of uint:

```
PublicBridge.sol
332:     for (uint i = 0; i < releasers.length; i++) {
374:     for (uint i = 0; i < releasers.length; i++) {
560:     for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
591:     for (uint i = 0; i < releases.length; i++) {
601:         for (uint i = 0; i < pendingRecipients.length; i++) {
612:         for (uint i = 0; i < releases.length; i++) {
620:         for (uint i = 0; i < newReleases.length; i++) {

PrivateChainBridge.sol
320:     for (uint i = 0; i < releasers.length; i++) {
376:     for (uint i = 0; i < releasers.length; i++) {
541:     for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
570:     for (uint i = 0; i < releases.length; i++) {
580:         for (uint i = 0; i < pendingRecipients.length; i++) {
591:         for (uint i = 0; i < releases.length; i++) {
599:         for (uint i = 0; i < newReleases.length; i++) {

Token.sol
7:     function transfer(address recipient, uint amount) external returns (bool);
8:     function transferFrom(address sender, address recipient, uint amount) external returns (bool);
9:     function approve(address spender, uint amount) external returns (bool);
12:     event Transfer(address indexed from, address indexed to, uint value);
13:     event Approval(address indexed owner, address indexed spender, uint value);
```

BridgeX: Fixed in commit [97b6c65](#) for PublicBridge, PrivateChainBridge.

Cyfrin: Verified.

7.3.7 Token::setIssuer allows setting issuer to address(0), disabling minting

Description: Token::constructor validates that the issuer is non-zero, but setIssuer has no such check:

```
// Token.sol:51 - constructor validates
require(tokenIssuer != address(0)); // dev: invalid issuer

// Token.sol:92-98 - setIssuer does not
function setIssuer(address newIssuer)
external
only(owner) {
    issuer = newIssuer; // no zero-address check
    emit NewIssuer(issuer);
}
```

Setting issuer to address(0) disables minting since msg.sender can never equal address(0), and only the owner (not the issuer) can call setIssuer to recover. The Certora spec mintAlwaysRevertsAfterZeroIssuer in Token.spec confirms this behavior.

Impact: If the owner accidentally calls setIssuer with address(0), minting is permanently disabled. The PublicBridge::_releaseTokens mint fallback would also break, forcing all releases to depend entirely on vault balance. This may be intentional to disable minting post-distribution, but the inconsistency with the constructor check suggests an oversight.

Recommended Mitigation: If this behavior is not desired, add a zero-address check consistent with the constructor:

```
function setIssuer(address newIssuer) external only(owner) {
    require(newIssuer != address(0)); // dev: invalid issuer
    issuer = newIssuer;
    emit NewIssuer(issuer);
}
```

BridgeX: Fixed in commit [c0c2f38](#).

Cyfrin: Verified.

7.3.8 PublicBridge::lockTokens allows zero-amount bridge operations when bridge fee is zero

Description: When bridgeFee == 0, calling lockTokens with amount = 0 succeeds because the fee check is skipped entirely:

```
uint256 bridgeAmount = amount; // 0
if (bridgeFee > 0) { // false, skipped
    // ...
}
require(token.transferFrom(msg.sender, address(vault), bridgeAmount), "Transfer to vault failed");
// transferFrom with 0 amount succeeds on this Token
```

This emits a TokensLocked event with zero amount and increments _nonce, which could create misleading events for relayer nodes that monitor lock events.

Recommended Mitigation: Add a minimum amount check:

```
require(amount > 0, "Amount required");
```

BridgeX: Fixed in commit [a460520](#).

Cyfrin: Verified.

7.3.9 Removed releaser sign still count

Description: When Bridge::removeReleaser is called, the contract only sets isReleaser[_releaser] = false and removes the address from the releasers array. It does not clear or adjust any existing signature state. So for every release that the removed releaser had already signed, signatures[txHash][_releaser] stays true and signatureCount[txHash] is unchanged. Those past signatures remain valid and still count toward the threshold when other releasers call Bridge::signRelease.

Recommended Mitigation: When removing a releaser, do not change past signatures/signatureCount but document that removed releasers' existing signatures still count and that requiredSignatures / releaser set should be managed with that in mind, or introduce a way to invalidate in-progress releases that include the removed releaser's signature

BridgeX: Acknowledged; generally releasers will only be rotated when there are emergencies and the bridge is paused. Addressing this would add a degree of complexity for little gain so we will not implement at this time.

7.3.10 Sanity check correct chainId matches current chain in PublicBridge, PrivateChainBridge::_releaseTokens

Description: PublicBridge::_releaseTokens fetches the chainId for the input destinationChain but never verifies that the chainId matches the current chain. Consider adding in this validation to prevent a scenario where the releasers sign for the incorrect destinationChain:

```
function _releaseTokens(
    address to,
    uint256 amount,
    DestinationChain destinationChain
) internal returns (bool) {
+   uint256 chainId = getChainId(destinationChain);
+   require(chainId == block.chainid, "Invalid destination chain for this bridge");

    uint256 vaultBalance = token.balanceOf(address(vault));
    // ... rest unchanged but use `chainId` instead of calling `getChainId`
```

A similar fix should be implemented in PrivateChainBridge::_releaseTokens to enforce that destinationChain == DestinationChain.Private.

BridgeX: Fixed in commit [f9686e8](#).

Cyfrin: Verified.

7.4 Gas Optimization

7.4.1 Use immutable variables for non-upgradeable contracts whose storage is only set once in the constructor

Description: Use immutable variables for non-upgradeable contracts whose storage is only set once in the constructor:

- `PublicBridge::TokenVault::bridge`, `token`
- `PublicBridge::token`, `vault`
- `PrivateChainBridge::TokenVault::bridge`
- `PrivateChainBridge::vault`
- `Token::maxSupply`, `decimals`

BridgeX: Fixed in commits [0917ee5](#), [32882e0](#).

Cyfrin: Verified.

7.4.2 In Solidity don't initialize to default values

Description: In Solidity don't initialize to default values:

```
PublicBridge.sol
73:     bool public paused = false;
348:     for (uint i = 0; i < releasers.length; i++) {
389:         uint256 distributed = 0;
390:         for (uint i = 0; i < releasers.length; i++) {
576:         for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
606:         uint256 remainingCount = 0;
607:         for (uint i = 0; i < releases.length; i++) {
617:             for (uint i = 0; i < pendingRecipients.length; i++) {
626:                 uint256 index = 0;
628:                 for (uint i = 0; i < releases.length; i++) {
636:                     for (uint i = 0; i < newReleases.length; i++) {

PrivateChainBridge.sol
74:     bool public paused = false;
332:     for (uint i = 0; i < releasers.length; i++) {
387:         uint256 distributed = 0;
388:         for (uint i = 0; i < releasers.length; i++) {
553:         for (uint i = 0; i < releases.length && eligibleBridgeReleases[recipient] > 0; i++) {
581:         uint256 remainingCount = 0;
582:         for (uint i = 0; i < releases.length; i++) {
592:             for (uint i = 0; i < pendingRecipients.length; i++) {
601:                 uint256 index = 0;
603:                 for (uint i = 0; i < releases.length; i++) {
611:                     for (uint i = 0; i < newReleases.length; i++) {
```

BridgeX: Fixed in commit [bd98983](#).

Cyfrin: Verified.

7.4.3 Better storage packing

Description: In Solidity:

- reading and writing from storage is expensive
- the order of storage declaration is important as it results in more or less storage slots being used to store the same data

Achieve better storage packing by:

- struct PendingRelease - declare destinationChain, exists after recipient

```
PublicBridge.sol
112:     struct PendingRelease {

PrivateChainBridge.sol
103:     struct PendingRelease {
```

BridgeX: Fixed in commit [937ba7c](#).

Cyfrin: Verified.

7.4.4 For reentrancy protection use transient keyword or ReentrancyGuardTransient instead of declaring a storage variable

Description: For reentrancy protection use transient keyword or [ReentrancyGuardTransient](#) instead of declaring a storage variable:

```
PublicBridge.sol
67:     uint256 private _locked = 1;

PrivateChainBridge.sol
71:     uint256 private _locked = 1;
```

Both solutions require increasing the solidity version but this should be fine as the current version 0.8.18 is relatively old.

BridgeX: Acknowledged; some of our chains are a bit older and don't have TSTORE and other modern opcodes like that. We'll make a note of it because the chain we will be deploying this on first is more modern, but we tend to like to keep these contracts as consistent as possible for all brands.

7.4.5 Cache storage to avoid identical storage reads

Description: In Solidity reading from storage is expensive; cache storage to avoid identical storage reads when the value can't change:

- PublicBridge, PrivateChainBridge::removeReleaser - cache releasers.length
- PublicBridge, PrivateChainBridge::payReleaseFee - cache releaseFee, releasers.length
- PublicBridge, PrivateChainBridge::lockTokens - cache bridgeFee prior to the if check and use cached value, cache bridgeFeeReceiver inside the if check
- PublicBridge, PrivateChainBridge::signRelease - cache signatureCount[txHash]++
- PublicBridge::_releaseTokens - cache token, vault
- PrivateChainBridge::_releaseTokens - cache vault
- PublicBridge, PrivateChainBridge::_processPendingReleases - cache releases.length
- PublicBridge, PrivateChainBridge::_cleanPendingReleases - cache releases.length, pendingRecipients.length
- PublicBridge, PrivateChainBridge::forceProcessPendingRelease - cache release.recipient, release.amount
- PublicBridge, PrivateChainBridge::acceptOwnership - cache pendingOwner
- Token::confirmOwnership - instead of modifier only(pendingOwner), use an internal function that returns pendingOwner, then use that cached return value to set owner and emit event TransferOwnership

BridgeX: Fixed in commit [57230de](#).

Cyfrin: Verified.

7.4.6 In `PublicBridge`, `PrivateChainBridge::payReleaseFee` don't iterate over `releasers` array when `feePerReleaser` is zero

Description: In `PublicBridge`, `PrivateChainBridge::payReleaseFee` don't iterate over `releasers` array when `feePerReleaser` is zero:

```
// Distribute fees directly to releasers (skip failures to prevent DoS)
uint256 distributed = 0;

if(feePerReleaser > 0) {
    for (uint i = 0; i < releasers.length; i++) {
        (bool success, ) = payable(releasers[i]).call{value: feePerReleaser}("");
        if (success) {
            distributed += feePerReleaser;
        }
    }
}
```

Since reading from storage is expensive, there's no need to iterate over the entire `releasers` array in storage when `feePerReleaser == 0`.

BridgeX: Fixed in commit [cef83f7](#).

Cyfrin: Verified.

7.4.7 Use input value instead of defining additional variable when original input value doesn't need to be preserved

Description: Use input value instead of defining additional variable when original input value doesn't need to be preserved:

- `PublicBridge::lockTokens` - remove `bridgeAmount` and just use/modify `amount`

BridgeX: Fixed in commit [521ff55](#).

Cyfrin: Verified.

7.4.8 Use `msg.sender` instead of `owner` inside `onlyOwner` functions

Description: Use `msg.sender` instead of `owner` inside `onlyOwner` functions:

- `PublicBridge`, `PrivateChainBridge::transferOwnership`

BridgeX: Fixed in commit [cb04abb](#).

Cyfrin: Verified.

7.4.9 Use input parameter when emitting events instead of reading `storage` when value already known

Description: Use input parameter when emitting events instead of reading `storage` when value already known:

- `Token::transferOwnership` - emit `TransferOwnership(newOwner, false)`;
- `Token::setIssuer` - emit `NewIssuer(newIssuer)`;

BridgeX: Fixed in commit [a372b10](#).

Cyfrin: Verified.